

```

import java.util.ArrayList;
import java.util.List;

// =====
// Interface: OrderComponent (Component)
// Defines the common interface for both individual products
// (Leaf) and composite containers (Box). Every component in the
// order tree must be able to report its price.
// MUHAMMAD SAMUDERA BAGJA
// =====
interface OrderComponent {
    // Returns the price of this component.
    // For a Leaf, this is the product's own price.
    // For a Composite, this is the sum of all children's prices.
    double getPrice();
}

// =====
// Leaf: Product
// Represents a single, indivisible item in an order (e.g. a
// book, pen, or notebook). It has no children and simply
// returns its own price.
// =====
class Product implements OrderComponent {
    private String name;
    private double price;

    // Constructs a Product with the given name and price.
    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    // Returns the price of this individual product.
    @Override
    public double getPrice() {
        return price;
    }

    // Returns a string representation of this product.
    @Override
    public String toString() {
        return "Product[name=" + name + ", price=" + price + "];"
    }
}

// =====
// Composite: Box
// A container that can hold other OrderComponents – both
// individual Products (Leaf) and other Boxes (Composite).
// Its price is calculated by summing the prices of all its
// children, enabling recursive tree traversal.
// =====
class Box implements OrderComponent {

```

```

// List of child components (can be Products or other Boxes)
private List<OrderComponent> children = new ArrayList<>();

// Adds a component (Product or Box) to this box.
public void add(OrderComponent c) {
    children.add(c);
}

// Removes a component from this box.
public void remove(OrderComponent c) {
    children.remove(c);
}

// Returns the list of child components.
public List<OrderComponent> getChildren() {
    return children;
}

// Calculates the total price by recursively summing
// the prices of all children in this box.
@Override
public double getPrice() {
    double total = 0;
    for (OrderComponent child : children) {
        total += child.getPrice();
    }
    return total;
}
}

// =====
// Client
// The client works exclusively through the OrderComponent
// interface. It does not need to know whether the underlying
// object is a single Product or a nested Box – the Composite
// pattern makes them interchangeable.
// =====
class Client {
    private OrderComponent order;

    // Constructs a Client with the given order component.
    public Client(OrderComponent order) {
        this.order = order;
    }

    // Calculates the total price of the entire order by
    // delegating to the root component's getPrice() method.
    public double calculateTotalOrderPrice() {
        return order.getPrice();
    }
}

// =====
// Main – Demo

```

```

// Demonstrates the Composite Pattern by building an order
// consisting of individual products and nested boxes, then
// calculating prices uniformly through a single interface.
// =====
public class CompositePatternOrder {
    public static void main(String[] args) {
        // Create individual leaf products
        Product book = new Product("Book", 25000);
        Product pen = new Product("Pen", 5000);
        Product notebook = new Product("Notebook", 15000);
        Product ruler = new Product("Ruler", 8000);

        // Create a small box containing pen + notebook
        Box smallBox = new Box();
        smallBox.add(pen);
        smallBox.add(notebook);

        // Create the main box containing book + smallBox + ruler
        Box mainBox = new Box();
        mainBox.add(book);
        mainBox.add(smallBox);
        mainBox.add(ruler);

        // The client only knows about OrderComponent -
        // it doesn't care whether it's a Leaf or a Composite
        Client client = new Client(mainBox);

        // Print pricing breakdown
        System.out.println("=== Composite Pattern - Order Pricing ===");
        System.out.println("Book          : Rp " + book.getPrice());
        System.out.println("Pen          : Rp " + pen.getPrice());
        System.out.println("Notebook     : Rp " + notebook.getPrice());
        System.out.println("Ruler        : Rp " + ruler.getPrice());
        System.out.println("-----");
        System.out.println("Small Box    : Rp " + smallBox.getPrice());
        System.out.println("Total Order  : Rp " + client.calculateTotalOrderPrice...
    }
}

```